PROJECT (3): CLOUD INFRASTRUCTURE AS A CODE IAC

Team 7

Names	ID's
Ahmed Mohamed Elsayed Tabbash	20P1076
Yaman Abdelhamid Elewa	2002078
Ahmed Abdallah Nofull	22P0255
Ferass Ahmed Mostafa	23P0304

SUMMER 2026

Contents

COMPREHENSIVE TECHNICAL REPORT: HIGH AVAILABILITY CLOUD INFRASTRUCTURE AUTOMATION

1. Project Overview and Objectives

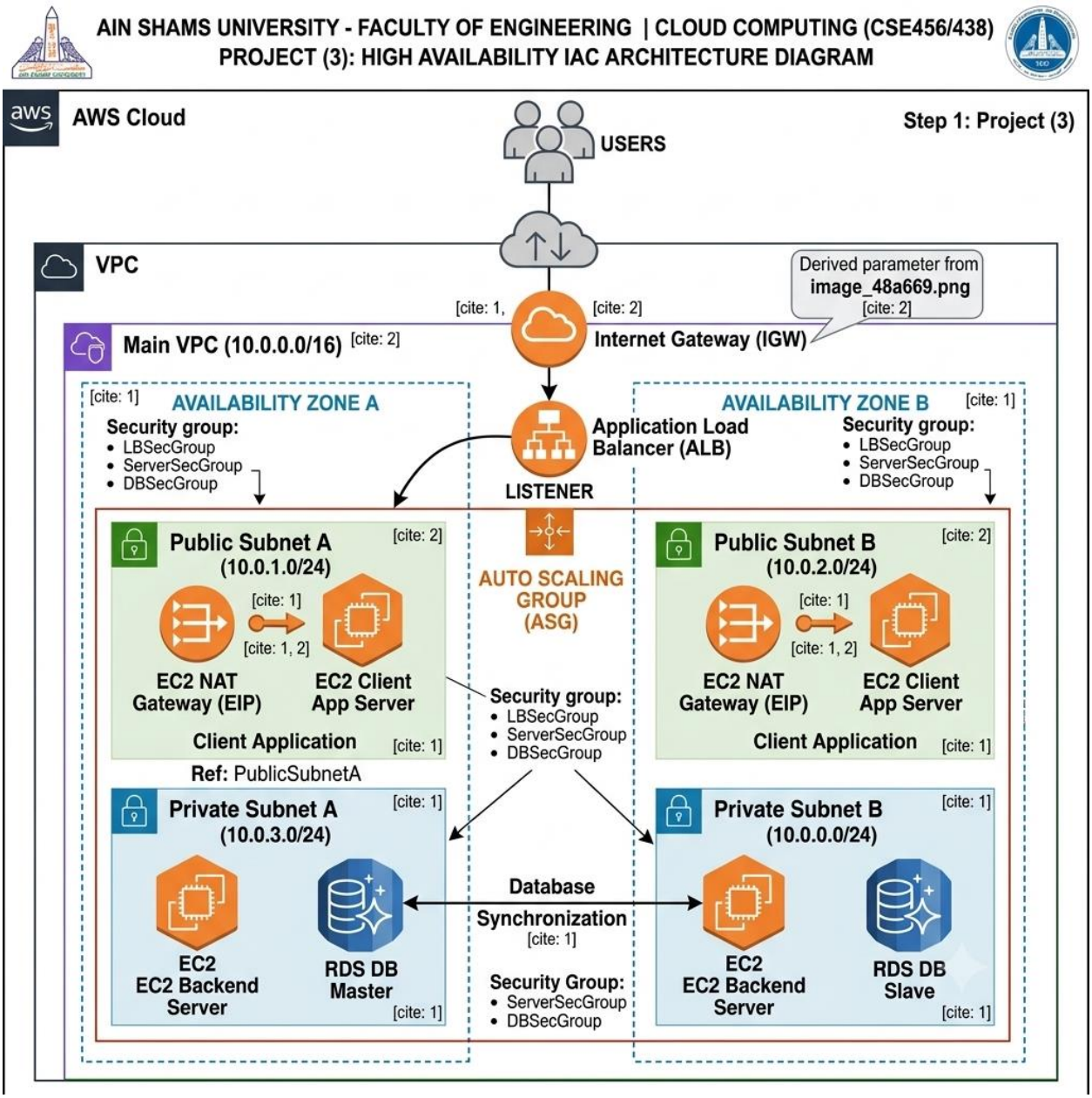
The primary objective of this project was to design, build, and automate a highly available, enterprise-ready cloud infrastructure using AWS CloudFormation. Moving away from manual infrastructure configuration, this implementation utilized Infrastructure as Code (IaC) to benefit from conventional software development practices. By treating infrastructure as software, the project achieved improved deployment speed, minimized human configuration errors, and eliminated configuration drift while ensuring consistent security strategies.

The target architecture required the automated creation of a Virtual Private Cloud (VPC), Public and Private subnets spanning multiple Availability Zones (AZs), NAT Gateways, an Internet Gateway (IGW), Elastic Compute Cloud (EC2) instances, and an Application Load Balancer (ALB).

2. Architecture Design Methodology

Before writing the IaC scripts, the network architecture was visually modeled using the Lucid Chart tool. To ensure fault tolerance and high availability, the architecture was explicitly divided across two distinct Availability Zones: Zone A and Zone B.

- **Public Tier:** Public Subnet A and Public Subnet B were provisioned to host outward-facing resources, specifically the Application Load Balancer and NAT Gateways. Client applications interface with this tier first.
- **Private Tier:** Private Subnet A and Private Subnet B were designed to secure the backend servers, completely isolating them from direct inbound internet access.



3. Parameterization Strategy (params.json)

To facilitate reusability and maintainability, the infrastructure configuration was separated from the declarative code using a JSON parameter file. This file acts as the single source of truth for the network's Classless Inter-Domain Routing (CIDR) blocks and for the size of the compute nodes.

Parameter Key	Assigned Value	Description
EnvironmentName	HA-Project	The global prefix used for tagging and identifying stack resources.
VpcNetCIDR	10.0.0.0/16	The overarching IP range for the entire Virtual Private Cloud.
PubSubnet01CIDR	10.0.1.0/24	IP range allocated to Public Subnet A in Availability Zone 1.
PubSubnet02CIDR	10.0.2.0/24	IP range allocated to Public Subnet B in Availability Zone 2.
PrivSubnet01CIDR	10.0.3.0/24	IP range allocated to Private Subnet A in Availability Zone 1.
PrivSubnet02CIDR	10.0.4.0/24	IP range allocated to Private Subnet B in Availability Zone 2.
InstanceType	t3.micro	The EC2 instance type used for the web servers, constrained by an AllowedValues list.
LatestAmiId	SSM parameter	Resolves at deployment time to the latest Amazon Linux 2 AMI in the target region, keeping the template region independent.

4. Network Infrastructure Automation (infrastructure.yaml)

The foundational network layer was automated using AWS CloudFormation in YAML format. The following components were programmatically provisioned:

- **Virtual Private Cloud (VPC):** Configured with DNS support and hostnames enabled to act as the secure container for all subsequent resources.
- **Subnet Allocation:** Four subnets were created utilizing the !Select and !GetAZs intrinsic functions to ensure they were evenly distributed across the region's available zones. Public IP mapping on launch was explicitly disabled across all subnets to enforce security compliance.
- **Internet Gateway (IGW):** Attached directly to the VPC to act as the primary node for external internet communication.
- **NAT Gateways & Elastic IPs (EIP):** Two NAT Gateways were provisioned—one in each public subnet—and assigned dedicated Elastic IPs. This allows resources in the private subnets to securely download updates without exposing their private IP addresses.
- **Route Tables:** A public route table was created with a 0.0.0.0/0 route pointing to the IGW, shared by both public subnets. Two distinct private route tables were created, each directing outbound traffic (0.0.0.0/0) to their respective AZ's NAT Gateway.

5. Compute, Load Balancing, and Auto-Scaling Layer

To handle variable application loads and ensure business continuity, the compute layer was dynamically configured.

- **Security Groups:** Deployed as virtual firewalls. The LBSecGroup allows inbound HTTP (Port 80) traffic from the public internet. The ServerSecGroup restricts backend EC2 access strictly to traffic originating from the Load Balancer.
- **Application Load Balancer (ALB): Provisioned across both public subnets to distribute incoming client traffic evenly. A Target Group and Listener were configured to perform continuous health checks on the backend instances, and a stack Output named WebAppLoadBalancerDNSName exposes the load balancer public DNS name.**

Launch Template & User Data: The deprecated Launch Configuration was replaced with a modern EC2 Launch Template. The AMI is resolved at deployment time from the AWS Systems Manager public parameter for the latest Amazon Linux 2 image, so the template is region independent. A bash script injected through the UserData property installs the Apache web server (httpd) and publishes the team landing page, so the Load Balancer health checks return healthy. Because the servers sit in the private subnets and reach the package repositories only through the NAT gateways, the script retries the installation until the NAT route is carrying traffic, and the Auto-Scaling Group declares an explicit DependsOn on the private routes so that no instance is launched before its route to the internet exists.

- **Auto-Scaling Group (ASG):** Configured with a minimum capacity of 2 and a maximum of 3 instances. The ASG spans both private subnets and is directly bound to the ALB's Target Group to ensure traffic is only routed to healthy nodes.

6. Deployment Execution and Testing

The infrastructure stack was validated and deployed through the AWS CloudFormation Management Console, where the template and the parameter values were submitted directly. The same deployment can equally be triggered from the AWS Command Line Interface (CLI) using the following command:

aws cloudformation create-stack --stack-name HA-Project-Stack --region us-east-1 --template-body file://infrastructure.yaml --parameters file://params.json

6.1 Stack Creation

The template and parameter file were submitted to AWS CloudFormation in the us-east-1 (N. Virginia) region. CloudFormation created all resources in the correct dependency order, and the deployment completed successfully with the stack reaching the CREATE_COMPLETE state.

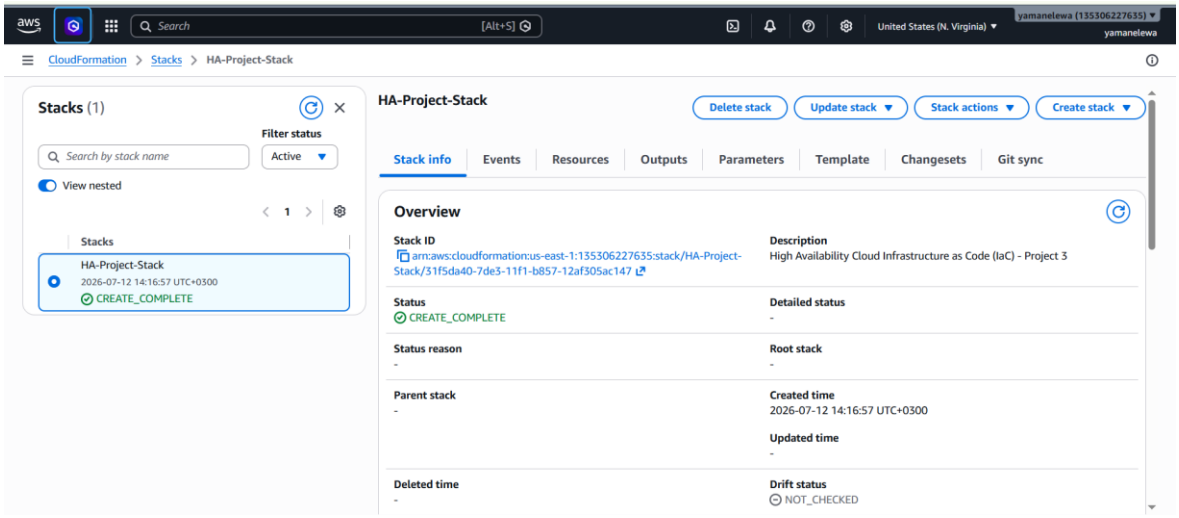


Figure 1. Stack creation completed successfully.

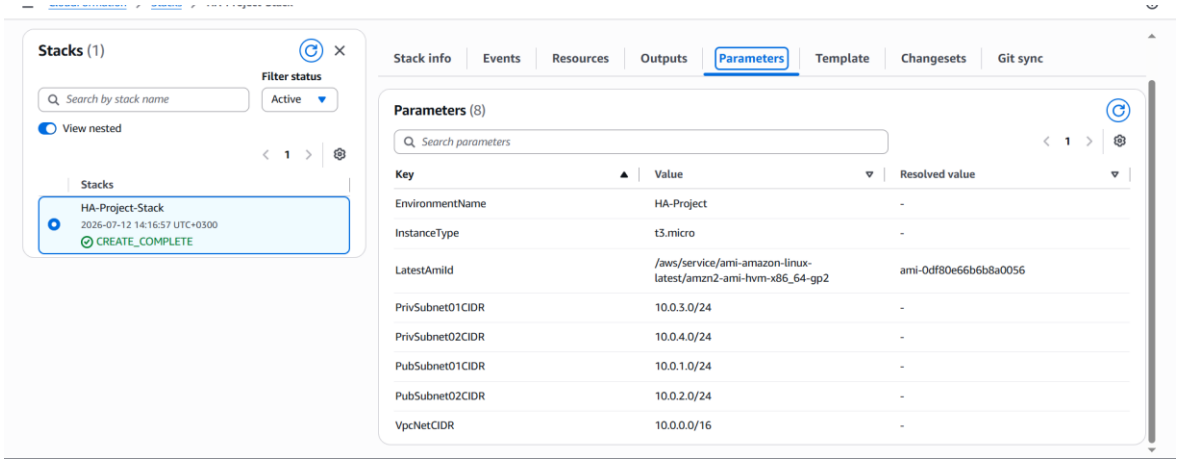


Figure 2. CloudFormation input parameters.

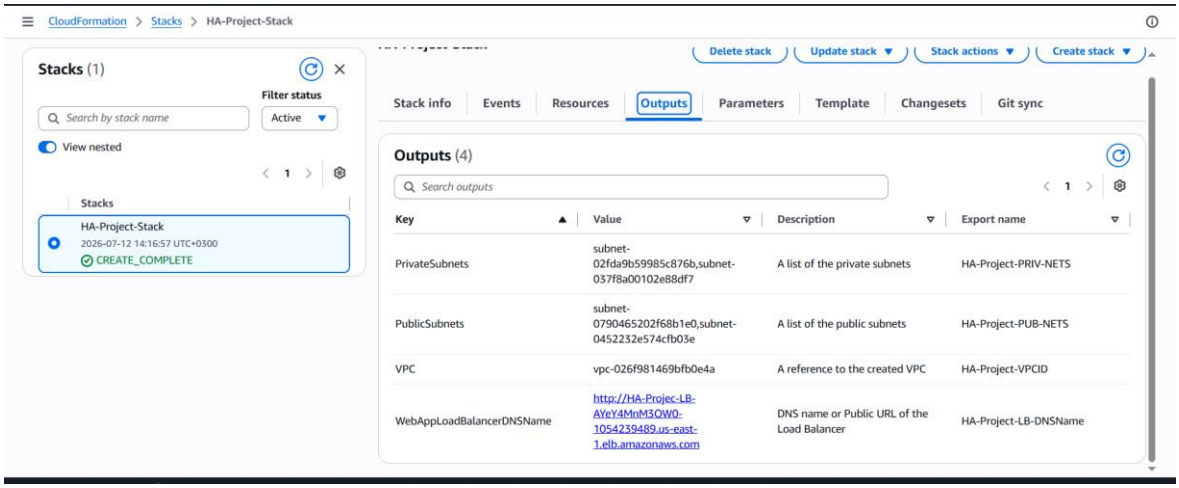


Figure 3. CloudFormation stack outputs.

6.2 Verification of the Deployed Website

The WebAppLoadBalancerDNSName output provides the public URL of the Application Load Balancer. Browsing to that address returned the web page served by the EC2 instances in the private subnets, confirming that the full request path is functional: the client reaches the internet-facing Load Balancer in the public subnets, which forwards the request over the Target Group to a healthy web server in a private subnet.

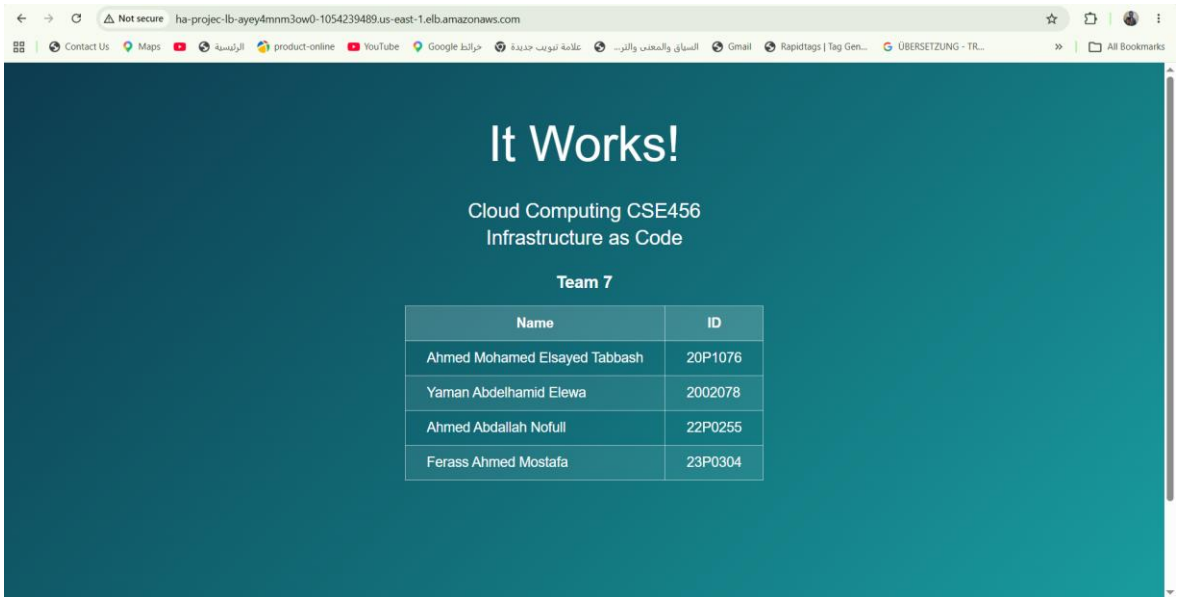


Figure 4. The website served through the Load Balancer.

6.3 High Availability Test: Auto-Scaling Self-Healing

To verify that the infrastructure could recover from a failure, one of the running EC2 instances was deliberately terminated. The Auto Scaling Group detected that the number of running instances had dropped below the minimum capacity of two and automatically launched a replacement instance in the same Availability Zone. The new instance was then registered with the Target Group and started serving traffic without any manual intervention.

During this test, the DNS name of the Load Balancer remained unchanged. Since the Load Balancer provides a fixed entry point to the application, the website continued to be accessible using the same URL while the failed instance was replaced. This demonstrates that the architecture can maintain service availability even when an EC2 instance fails.

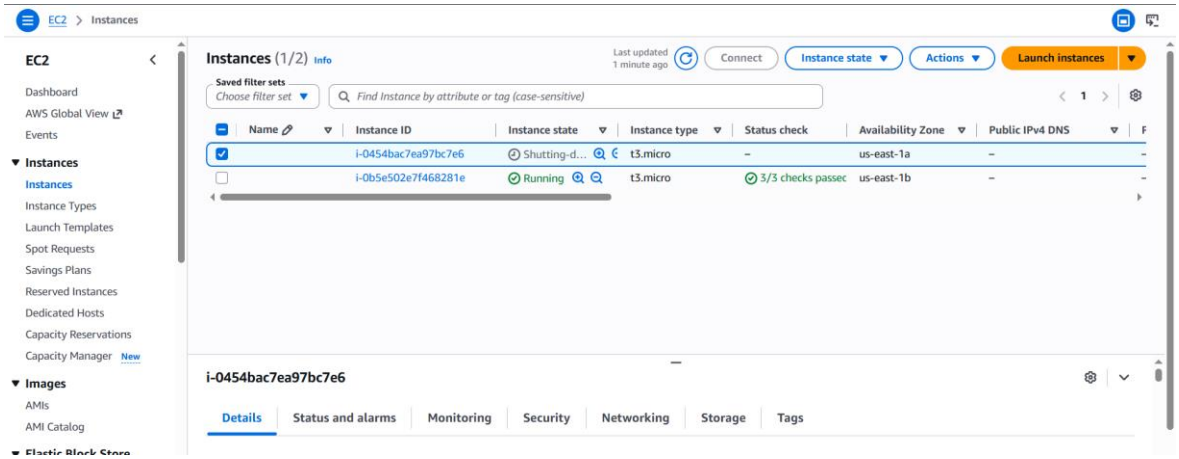


Figure 5. Terminated EC2 instance during the failover test.

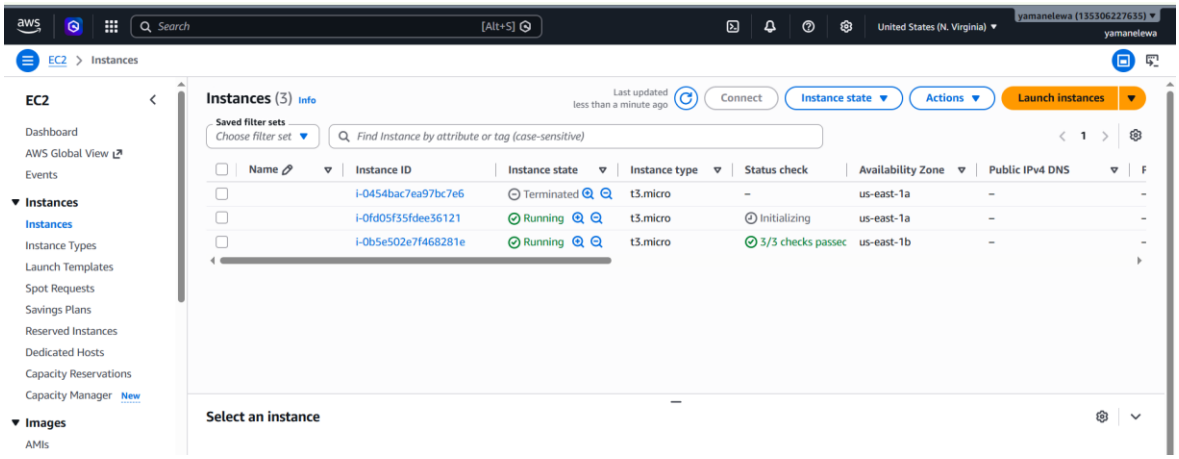


Figure 6. Replacement EC2 instance launched by Auto Scaling.

6.4 Stack Deletion

Finally, the entire infrastructure was deleted using a single CloudFormation delete operation. CloudFormation removed all resources in the correct dependency order, and the stack reached the DELETE_COMPLETE state, leaving no remaining infrastructure or unnecessary costs.

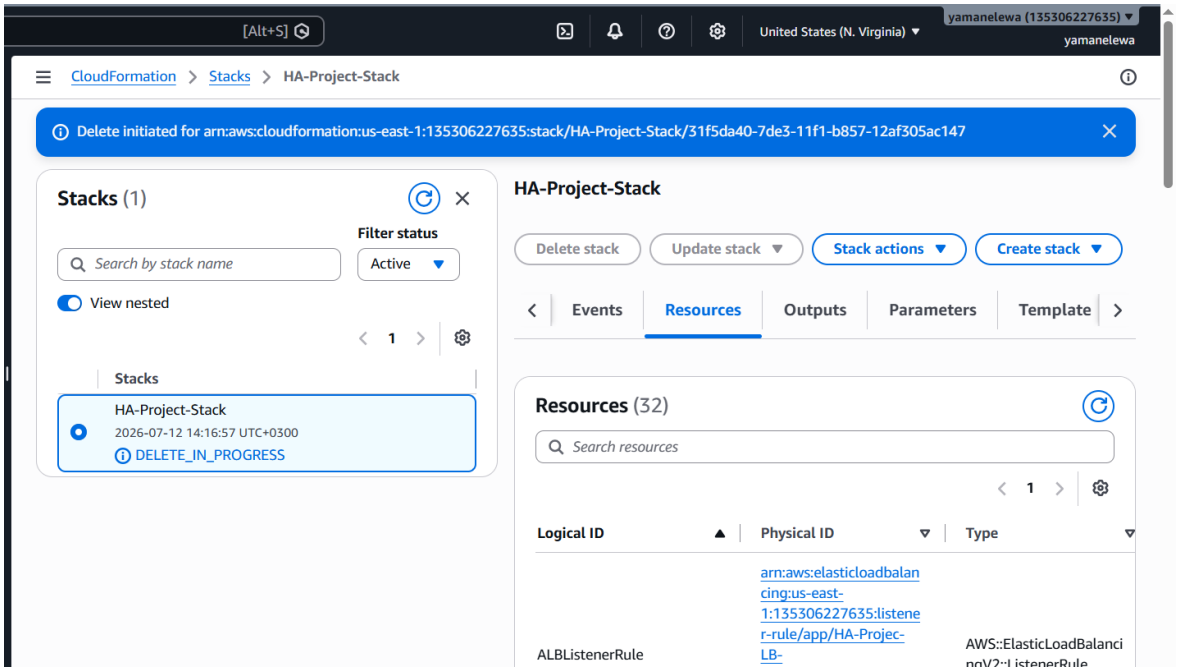


Figure 7. Stack deletion in progress.

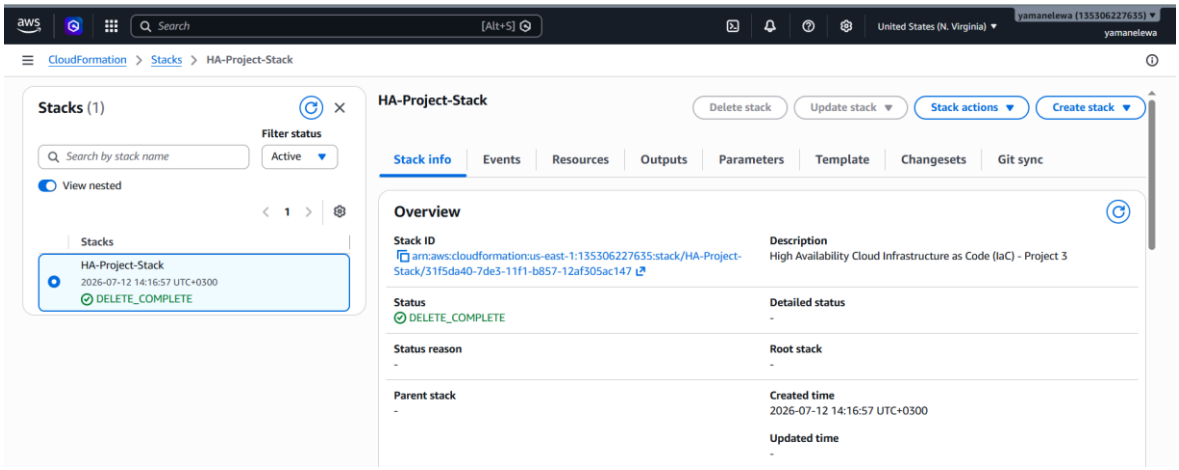


Figure 8. Stack deletion completed successfully.

7. Conclusion

The project successfully demonstrated the benefits of Infrastructure as Code (IaC). Using declarative YAML and JSON files, a highly available cloud infrastructure was deployed automatically through AWS CloudFormation. The resulting infrastructure meets the project requirements by distributing traffic across multiple Availability Zones while securely hosting the application within the AWS environment.